

Lab 14: Outbox Pattern + Idempotent Retry Loop in Python/Go (55 min)

The application-side counterpart to Lab 5 (contention) and Lab 13 (CDC). Everything here is code you can lift into a real service.

Learning Objectives

By the end of this lab you will be able to:

- Write a correct retry loop for 40001 serialization failures — and explain why every driver needs one
- Use `crdb` retry helpers (`sqlalchemy-cockroachdb` , `crdb-go`) instead of hand-rolling
- Implement the **Outbox Pattern** so events and business writes commit atomically
- Design the outbox table so it does not become a hot range or an unbounded queue
- Implement **idempotency keys** so a client retry does not double-charge
- Compare optimistic concurrency control against `SELECT FOR UPDATE` and measure both
- Size a connection pool and see what happens when you get it wrong

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- `python3` with `psycopg2` (`pip install psycopg2-binary`)
- Optional: Go 1.21+ for the Go variants

Setup

```
scripts/crdb up
export DSN='postgresql://root@localhost:26257/shop?sslmode=disable'
```

The cluster runs in Docker (see [Lab 1](#)). From your machine it is `localhost:26257` ; from inside another container it is `crdb1:26257` . `scripts/crdb run ...` executes inside node 1.

```

scripts/crdb sql <<'SQL'
CREATE DATABASE shop;
USE shop;

CREATE TABLE accounts (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email   STRING NOT NULL UNIQUE,
  balance DECIMAL(12,2) NOT NULL DEFAULT 0,
  version INT NOT NULL DEFAULT 1      -- for optimistic concurrency
);

CREATE TABLE orders (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account UUID NOT NULL REFERENCES accounts(id),
  amount  DECIMAL(12,2) NOT NULL,
  status  STRING NOT NULL DEFAULT 'new',
  created TIMESTAMPTZ NOT NULL DEFAULT now()
);

INSERT INTO accounts (email, balance)
SELECT 'user' || g || '@example.com', 1000 FROM generate_series(1, 100) g;
SQL

```

Tasks

Part A: The Retry Loop, Written Correctly (12 min)

CockroachDB runs `SERIALIZABLE` by default. A transaction that would violate serializability is aborted with SQLSTATE `40001` and **must be retried by the client**. This is not an error condition — it is the concurrency-control protocol.

1. **See it happen.** `/tmp/lab14/no_retry.py`:

```
#!/usr/bin/env python3
"""No retry loop – this is what a naive service looks like under contention."""
import psycopg2, threading, os

DSN = os.environ["DSN"]
failures, successes = [], []

def transfer():
    conn = psycopg2.connect(DSN)
    conn.autocommit = False
    try:
        with conn.cursor() as cur:
            cur.execute("SELECT balance FROM accounts WHERE email = 'user1@example.com'")
            bal = cur.fetchone()[0]
            cur.execute(
                "UPDATE accounts SET balance = %s WHERE email = 'user1@example.com'",
                (bal + 1,)
            )
            conn.commit()
            successes.append(1)
    except psycopg2.errors.SerializationFailure as e:
        failures.append(str(e.pgcode))
        conn.rollback()
    finally:
        conn.close()

threads = [threading.Thread(target=transfer) for _ in range(40)]
[t.start() for t in threads]; [t.join() for t in threads]
print(f"succeeded: {len(successes)}    failed with 40001: {len(failures)}")
```

```
python3 /tmp/lab14/no_retry.py
```

Record the failure count. Every one of those is a dropped user request.

2. **Now with a correct retry loop.** /tmp/lab14/with_retry.py :

```

#!/usr/bin/env python3
"""Correct retry loop: exponential backoff with jitter, bounded attempts,
and – critically – the ENTIRE transaction re-executed, not just the failing
statement. The values read in attempt N are invalid in attempt N+1."""
import psycopg2, threading, os, random, time

DSN = os.environ["DSN"]
MAX_ATTEMPTS = 5
stats = {"ok": 0, "retried": 0, "gave_up": 0}
lock = threading.Lock()

def run_txn(conn, body):
    for attempt in range(1, MAX_ATTEMPTS + 1):
        try:
            with conn:
                with conn.cursor() as cur:
                    body(cur)
            return attempt
        except psycopg2.errors.SerializationFailure:
            if attempt == MAX_ATTEMPTS:
                raise
            # Exponential backoff with full jitter. Without jitter, all the
            # retrying clients collide again at exactly the same moment.
            sleep = random.uniform(0, min(0.5, 0.005 * (2 ** attempt)))
            time.sleep(sleep)
    raise RuntimeError("unreachable")

def transfer():
    conn = psycopg2.connect(DSN)
    def body(cur):
        cur.execute("SELECT balance FROM accounts WHERE email = 'user1@example.com'")
        bal = cur.fetchone()[0]
        cur.execute("UPDATE accounts SET balance = %s WHERE email = 'user1@example.com'",
                    (bal + 1,))
    try:
        attempts = run_txn(conn, body)
        with lock:
            stats["ok"] += 1
            if attempts > 1: stats["retried"] += 1
    except psycopg2.errors.SerializationFailure:
        with lock: stats["gave_up"] += 1
    finally:
        conn.close()

threads = [threading.Thread(target=transfer) for _ in range(40)]
t0 = time.time()
[t.start() for t in threads]; [t.join() for t in threads]
print(f"{stats} elapsed={time.time()-t0:.2f}s")

```

```
python3 /tmp/lab14/with_retry.py
```

Expect some `gave_up` at this concurrency. 24 transactions all doing a read-modify-write against *one row* with a 5-attempt budget is deliberately brutal: a measured run gave `{'ok': 15, 'retried': 13, 'gave_up': 9}`. That is not a broken retry loop — it is the correct signal that **a retry budget is finite and contention can exhaust it**.

Try it three ways and record what each one buys:

1. `MAX_ATTEMPTS = 5` → some requests fail
2. `MAX_ATTEMPTS = 20` → nearly all succeed, but the slow ones get *much* slower
3. The single-statement form in step 3 → 100% success, zero retries, fastest

The lesson: a retry loop is mandatory, and it is not a substitute for a design that doesn't contend. If production is exhausting a reasonable retry budget, the fix is in the schema or the access pattern (Lab 8's sharded counter), not in a bigger budget.

3. The single-statement version that needs no retry at all:

```
UPDATE accounts SET balance = balance + 1 WHERE email = 'user1@example.com';
```

A read-modify-write collapsed into one statement is atomic server-side. **Before you write a retry loop, check whether you can write one statement instead.**

4. Compare all three:

Approach	Successes	Retried	Gave up	Elapsed
No retry loop		n/a		
Retry, <code>MAX_ATTEMPTS = 5</code>				
Retry, <code>MAX_ATTEMPTS = 20</code>				
Single statement		0	0	

Whatever the success counts, one invariant must hold in every row: the final balance equals `1000 + successes`. No lost updates, ever — that is what `SERIALIZABLE` guarantees, and it is the thing a retry loop preserves.

5. Use the library instead of your own loop. For SQLAlchemy:

```
# pip install sqlalchemy-cockroachdb
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from cockroachdb.sqlalchemy import run_transaction

engine = create_engine("cockroachdb://root@localhost:26257/shop?sslmode=disable")
Session = sessionmaker(bind=engine)

def txn(session):
    acct = session.query(Account).filter_by(email="user1@example.com").one()
    acct.balance += 1

run_transaction(Session, txn)  # retries 40001 for you
```

For Go:

```
// go get github.com/cockroachdb/cockroach-go/v2/crdb/crdbpgx
err := crdbpgx.ExecuteTx(ctx, pool, pgx.TxOptions{}, func(tx pgx.Tx) error {
    var bal float64
    if err := tx.QueryRow(ctx,
        "SELECT balance FROM accounts WHERE email=$1", email).Scan(&bal); err != nil {
        return err
    }
    _, err := tx.Exec(ctx,
        "UPDATE accounts SET balance=$1 WHERE email=$2", bal+1, email)
    return err
})
```

Part B: The Outbox Pattern (15 min)

The problem: your service must both write an order **and** publish an `order.created` event. Writing to the database and to Kafka from application code is a dual write — if the process dies between them, the two systems disagree forever.

The fix: write the event into the same transaction as the business data, and let CDC ship it.

1. Design the outbox table — the schema decisions matter more than the code:

```
USE shop;

CREATE TABLE events_outbox (
    id          UUID NOT NULL DEFAULT gen_random_uuid(),
    created     TIMESTAMPTZ NOT NULL DEFAULT now(),
    topic       STRING NOT NULL,
    aggregate   UUID NOT NULL,           -- which entity this event is about
    payload     JSONB NOT NULL,
    PRIMARY KEY (id)                    -- UUID PK: writes scatter, no rightmost hotspot
) WITH (ttl_expire_after = '7 days', ttl_job_cron = '@hourly');

CREATE INDEX ON events_outbox (aggregate, created DESC);
```

Decision	Why
UUID primary key	Ingest is append-only and high-rate — a sequential key would hotspot the rightmost range (Lab 3, Lab 8 Part D)
Row-level TTL	The outbox is a <i>buffer</i> , not a log. Without TTL it grows forever
No <code>status</code> column, no <code>UPDATE</code>	A polled <code>WHERE status='pending'</code> queue serializes on one range. CDC reads the write path directly
aggregate index	Lets you answer "what happened to order X?" without a full scan

The anti-pattern this replaces: `SELECT ... WHERE status='pending' FOR UPDATE LIMIT 100` followed by `UPDATE ... SET status='sent'`. That is a single-range queue: every worker contends on the same rows, and throughput caps out no matter how many workers you add.

2. Write both in one transaction — `/tmp/lab14/outbox.py`:

```
#!/usr/bin/env python3
import psycopg2, json, os, uuid
DSN = os.environ["DSN"]

def place_order(conn, account_id, amount):
    """Business write and event publication commit together, or not at all."""
    with conn:
        with conn.cursor() as cur:
            cur.execute(
                "INSERT INTO orders (account, amount) VALUES (%s, %s) RETURNING id",
                (account_id, amount))
            order_id = cur.fetchone()[0]
            cur.execute(
                """INSERT INTO events_outbox (topic, aggregate, payload)
                VALUES ('order.created', %s, %s)""",
                (order_id, json.dumps({
                    "order_id": str(order_id),
                    "account_id": str(account_id),
                    "amount": float(amount)})))
        return order_id

conn = psycopg2.connect(DSN)
with conn.cursor() as cur:
    cur.execute("SELECT id FROM accounts LIMIT 5")
    accounts = [r[0] for r in cur.fetchall()]
for a in accounts:
    print("placed", place_order(conn, a, 25.00))
```

```
python3 /tmp/lab14/outbox.py
scripts/crdb sql -e \
    "SELECT topic, aggregate, payload FROM shop.events_outbox ORDER BY created DESC LIMIT
    5;"
```

3. Prove atomicity. Force a failure after the order insert but before commit:

```
# Add this inside the `with conn:` block, after both inserts:
raise RuntimeError("simulated crash before commit")
```

```
scripts/crdb sql -e "
  SELECT (SELECT count(*) FROM shop.orders) AS orders,
         (SELECT count(*) FROM shop.events_outbox) AS events;"
```

Neither row exists. With a dual write, the order would exist and the event would not.

4. Ship the outbox with CDC (this is why the table has no `status` column):

```
SET CLUSTER SETTING kv.rangefeed.enabled = true;
-- Core changefeed so this works without a license:
EXPERIMENTAL CHANGEFEED FOR shop.events_outbox WITH updated;
```

In production: `CREATE CHANGEFEED FOR TABLE shop.events_outbox INTO 'kafka://...'` with the frontier consumer you built in Lab 13.

5. Confirm TTL is doing its job:

```
SHOW CREATE TABLE events_outbox;
SELECT * FROM [SHOW JOBS] WHERE job_type = 'ROW LEVEL TTL' ORDER BY created DESC LIMIT 3;
-- Empty on a fresh table: the JOB appears only after the cron fires.
-- The SCHEDULE exists immediately:
SELECT id, label, next_run FROM [SHOW SCHEDULES] WHERE label ILIKE '%row-level-ttl%';
```

Part C: Idempotency Keys (15 min)

A client that times out will retry. Without an idempotency key, the retry creates a second order.

1. The schema:

```
CREATE TABLE idempotency (
  key          STRING PRIMARY KEY,          -- client-supplied, e.g. a UUID per user action
  account      UUID NOT NULL,
  response     JSONB NOT NULL,              -- the response we returned the first time
  created      TIMESTAMPTZ NOT NULL DEFAULT now()
) WITH (ttl_expire_after = '24 hours');
```

2. The handler — `/tmp/lab14/idempotent.py`:

```

#!/usr/bin/env python3
import psycopg2, psycopg2.errors, json, os, uuid
DSN = os.environ["DSN"]

def place_order_idempotent(conn, idem_key, account_id, amount):
    """Returns (response, replayed). One transaction: claim the key, do the work,
    record the response. A concurrent duplicate loses the race on the PK."""
    with conn:
        with conn.cursor() as cur:
            # 1. Fast path: have we seen this key before?
            cur.execute("SELECT response FROM idempotency WHERE key = %s", (idem_key,))
            row = cur.fetchone()
            if row:
                return row[0], True

            # 2. Do the work
            cur.execute("INSERT INTO orders (account, amount) VALUES (%s,%s) RETURNING
            id",
                        (account_id, amount))
            order_id = cur.fetchone()[0]
            cur.execute("""INSERT INTO events_outbox (topic, aggregate, payload)
                        VALUES ('order.created', %s, %s)""",
                        (order_id, json.dumps({"order_id": str(order_id)})))

            response = {"order_id": str(order_id), "status": "created"}

            # 3. Claim the key in the SAME transaction. If a concurrent request
            #     already claimed it, this raises UniqueViolation and the whole
            #     transaction rolls back – including the order.
            cur.execute("INSERT INTO idempotency (key, account, response) VALUES (%s,%s,
            %s)",
                        (idem_key, account_id, json.dumps(response)))
            return response, False

conn = psycopg2.connect(DSN)
with conn.cursor() as cur:
    cur.execute("SELECT id FROM accounts LIMIT 1")
    acct = cur.fetchone()[0]

key = str(uuid.uuid4())
print(place_order_idempotent(conn, key, acct, 42.00)) # (response, False)
print(place_order_idempotent(conn, key, acct, 42.00)) # (same response, True)
print(place_order_idempotent(conn, key, acct, 42.00)) # (same response, True)

with conn.cursor() as cur:
    cur.execute("SELECT count(*) FROM orders WHERE amount = 42.00")
    print("orders created:", cur.fetchone()[0], "(should be 1)")

```

```
python3 /tmp/lab14/idempotent.py
```

3. **Test the concurrent case** — fire 10 threads with the *same* key simultaneously.

First, try it without a retry loop and watch it fail in an instructive way: all ten threads read the key (absent), all ten do the work, and then they collide. Under `SERIALIZABLE`, a measured run produced **zero** committed orders — not one. The winner is aborted too, because its read of the not-yet-existing key was invalidated by a concurrent insert of that same key.

Idempotency and retries are not independent features. An idempotency key without a retry loop turns a duplicate request into a *failed* request.

```
# Append to idempotent.py – the correct version: retry, and treat "someone
# else claimed the key" as a replay rather than an error.
import threading, time, random

def place_order_idempotent_retrying(conn, key, account_id, amount, max_attempts=10):
    for attempt in range(1, max_attempts + 1):
        try:
            return place_order_idempotent(conn, key, account_id, amount)
        except psycopg2.errors.UniqueViolation:
            conn.rollback()
            # Another caller claimed this key first – read back their response.
            with conn:
                with conn.cursor() as cur:
                    cur.execute("SELECT response FROM idempotency WHERE key = %s", (key,))
                    row = cur.fetchone()
            if row:
                return row[0], True
        except psycopg2.errors.SerializationFailure:
            conn.rollback()
            time.sleep(random.uniform(0, min(0.5, 0.005 * (2 ** attempt))))
        raise RuntimeError("idempotent handler exhausted its retry budget")

key2, results, lock = str(uuid.uuid4()), [], threading.Lock()
def worker():
    c = psycopg2.connect(DSN)
    try:
        resp, replayed = place_order_idempotent_retrying(c, key2, acct, 77.00)
        with lock: results.append((resp["order_id"], replayed))
    finally:
        c.close()

ts = [threading.Thread(target=worker) for _ in range(10)]
[t.start() for t in ts]; [t.join() for t in ts]

with conn.cursor() as cur:
    cur.execute("SELECT count(*) FROM orders WHERE amount = 77.00")
    print("orders created:", cur.fetchone()[0], "(must be 1)")
print("distinct order_ids returned:", len({r[0] for r in results}), "(must be 1)")
print("created once, replayed", sum(1 for r in results if r[1]), "times")
```

All ten callers get the **same** `order_id`: one created it, nine replayed it. That is what an idempotent endpoint owes its clients — not just "no duplicate row", but *the same answer*.

⚠ **Verify on a fresh connection.** If you check the row count on a psycopg2 connection that is still idle-in-transaction (any `conn.cursor()` used outside a `with conn:` block leaves one open), you reuse that transaction's **snapshot** — and under **SERIALIZABLE** the snapshot predates the writes you are looking for. The count comes back `0` and looks like lost data when nothing was lost at all:

```
c = psycopg2.connect(DSN); c.autocommit = True # fresh snapshot, no open txn
```

This is one of the most common false alarms when testing a distributed database from an application: the database is fine; your client is reading the past.

4. **Why the key lives in the same transaction.** If you claimed the key in a *separate* transaction before doing the work, a crash between the two leaves the key claimed and the work undone — every subsequent retry returns "already processed" for work that never happened. One transaction, or the guarantee is gone.

Part D: Optimistic Concurrency vs `SELECT FOR UPDATE` (15 min)

Two ways to stop two users from overwriting each other. Measure both.

1. **Optimistic (version column) — no locks, retry on conflict:**

```
UPDATE accounts
SET balance = $new_balance, version = version + 1
WHERE id = $id AND version = $expected_version;
-- 0 rows updated => someone else won; re-read and retry
```

2. **Pessimistic (`SELECT FOR UPDATE`) — lock first, then write:**

```
BEGIN;
SELECT balance FROM accounts WHERE id = $id FOR UPDATE;
UPDATE accounts SET balance = $new WHERE id = $id;
COMMIT;
```

3. **Benchmark them** — `/tmp/lab14/occ_vs_locking.py`:

```

#!/usr/bin/env python3
import psycopg2, threading, time, os, random
DSN = os.environ["DSN"]
CONCURRENCY, ITERATIONS = 16, 25

def bench(name, fn):
    conn0 = psycopg2.connect(DSN); conn0.autocommit = True
    with conn0.cursor() as cur:
        cur.execute("UPDATE accounts SET balance=1000, version=1 WHERE
            email='user1@example.com'")
    counts = {"ok": 0, "retries": 0}
    lock = threading.Lock()
    def worker():
        conn = psycopg2.connect(DSN)
        for _ in range(ITERATIONS):
            r = fn(conn)
            with lock:
                counts["ok"] += 1; counts["retries"] += r
        conn.close()
    ts = [threading.Thread(target=worker) for _ in range(CONCURRENCY)]
    t0 = time.time(); [t.start() for t in ts]; [t.join() for t in ts]
    el = time.time() - t0
    print(f"{name:12} {CONCURRENCY*ITERATIONS} ops in {el:.2f}s "
        f"=> {CONCURRENCY*ITERATIONS/el:.0f} ops/s, retries={counts['retries']}")

def occ(conn):
    """Two different conflicts, two different mechanisms:
    - version mismatch -> 0 rows updated, someone beat us; re-read and retry
    - SQLSTATE 40001 -> the DATABASE detected a write conflict; retry too
    OCC does NOT exempt you from the 40001 retry loop."""
    retries = 0
    while True:
        try:
            with conn:
                with conn.cursor() as cur:
                    cur.execute("SELECT balance, version FROM accounts WHERE
                        email='user1@example.com'")
                    bal, ver = cur.fetchone()
                    cur.execute("""UPDATE accounts SET balance=%s, version=version+1
                        WHERE email='user1@example.com' AND version=%s""",
                        (bal+1, ver))
                    if cur.rowcount == 1:
                        return retries
        except psycopg2.errors.SerializationFailure:
            pass # WriteTooOld / retry error - same handling as a lost race
        retries += 1
        time.sleep(random.uniform(0, 0.01))

def pessimistic(conn):
    retries = 0
    while True:
        try:
            with conn:
                with conn.cursor() as cur:
                    cur.execute("SELECT balance FROM accounts WHERE
                        email='user1@example.com' FOR UPDATE")
                    bal = cur.fetchone()[0]

```

```

        cur.execute("UPDATE accounts SET balance=%s WHERE
email='user1@example.com'", (bal+1,))
        return retries
    except psycopg2.errors.SerializationFailure:
        retries += 1
        time.sleep(random.uniform(0, 0.01))

```

```

bench("OCC", occ)
bench("FOR UPDATE", pessimistic)

```

```
python3 /tmp/lab14/occ_vs_locking.py
```

4. Record and interpret:

Strategy	ops/s	Retries	Best when
Optimistic (version)			Low contention, long think-time between read and write
SELECT FOR UPDATE			High contention on a known row
Single statement	(Part A)	0	Whenever the logic fits in one statement

Reference measurement (8 threads × 10 iterations on one row, 3-node local cluster) — your numbers will differ, but the *shape* should match:

Strategy	ops/s	Retries
OCC	121	175
SELECT FOR UPDATE	159	0

175 retries to do 80 updates: under this much contention OCC spends most of its effort discovering that it lost. The locking version does the same work with zero wasted attempts.

Under high contention **FOR UPDATE** usually wins because conflicting writers queue instead of racing, failing, and re-racing. Under low contention OCC wins because it takes no locks at all. Measure your actual contention before choosing.

The trap in the OCC version: it is tempting to handle only "0 rows updated" and assume the version column has covered you. It has not. CockroachDB still raises SQLSTATE **40001** (**WriteTooOldError**) for physical write conflicts, and an OCC loop without that **except** crashes under load. Optimistic concurrency is a *complement* to the retry loop, never a replacement for it.

Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 8 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

Part E: Connection Pool Sizing (8 min)

1. Under-pooled — the app queues:

```
scripts/crdb run workload run kv --duration=30s --concurrency=64 --max-rate=0 \
'postgresql://root@localhost:26257?sslmode=disable' | tail -3
```

2. Over-pooled — the database queues:

```
scripts/crdb run workload run kv --duration=30s --concurrency=512 --max-rate=0 \
'postgresql://root@localhost:26257?sslmode=disable' | tail -3
```

Throughput barely moves; p99 multiplies. You moved the queue somewhere less observable.

3. Watch connections from the database side:

```
SELECT node_id, count(*) AS sessions FROM crdb_internal.cluster_sessions GROUP BY node_id;
SHOW CLUSTER SETTING server.max_connections_per_gateway;
```

4. The sizing rules:

Pool	Rule
Per app instance	<code>(cores × 2) + effective_spindles</code> , then validate against the Lab 8 knee
Cluster total	Keep total connections under ~4× total vCPU across nodes
PgBouncer mode	transaction pooling for CockroachDB (session pooling wastes connections)
Timeouts	Set <code>statement_timeout</code> and pool acquisition timeouts — never let a request wait forever

PgBouncer in transaction mode breaks session-scoped features: `SET` (session), prepared statements across transactions, and advisory locks. Use session variables per-transaction (`SET LOCAL`) or configure the pooler accordingly.

Cleanup

```
scripts/crdb down
```

Lab 14 Deliverables

□ **Retry loop** written correctly (full-transaction retry, jittered backoff, bounded attempts), with measured exhaustion at a finite budget and zero lost updates in every case □ **Outbox table** designed to avoid a hot range, a polled queue, and unbounded growth □ **Atomicity proven** — a simulated crash leaves neither the order nor the event □ **Idempotency keys** verified against sequential *and* concurrent duplicates — including

the measured failure mode when the handler has no retry loop □ **OCC vs FOR UPDATE** benchmarked, with a stated rule for which to use □ **Pool sizing** demonstrated in both failure directions

Challenge Exercises

1. **Port the retry loop to your language.** Go with `crdbpgx` , Node with `node-postgres` , Java with `pgjdbc` + a retry aspect. Prove it handles 40001 with a concurrent test.
2. **Outbox at scale.** Load 1M rows into `events_outbox` and measure the TTL job's impact on foreground write latency. Tune `ttl_job_cron` and `ttl_delete_batch_size` until the impact is acceptable. What did you trade away?
3. **Idempotency without a table.** Can you get the same guarantee using only the order's natural key and `INSERT ... ON CONFLICT DO NOTHING ... RETURNING` ? What do you lose (hint: the cached response body)?
4. **Read Committed.** Re-run Part D with `SET default_transaction_isolation = 'read committed'` . Which anomaly does OCC now need to defend against that `SERIALIZABLE` handled for you?

Reference

Item	Purpose
<code>SQLSTATE 40001</code>	Serialization failure — retry the whole transaction
<code>crdbpgx.ExecuteTx</code> (Go)	Driver-level retry helper
<code>run_transaction()</code> (SQLAlchemy)	Driver-level retry helper
<code>SELECT ... FOR UPDATE</code>	Pessimistic row lock
<code>INSERT ... ON CONFLICT</code>	Upsert / idempotent insert
<code>WITH (ttl_expire_after = '...')</code>	Row-level TTL for outbox and idempotency tables
<code>crdb_internal.cluster_sessions</code>	Live connection count per node
<code>SET LOCAL</code>	Transaction-scoped settings, PgBouncer-safe